

GESTIÓN Y TICS

SENA Portal

MANUAL TÉCNICO

Documentación de arquitectura, configuración y mantenimiento del sistema

Versión 1.0 | Mayo 2026

Fabian Andrés Mabesoy

Angelica María Méndez

Servicio Nacional de Aprendizaje — SENA

1. Introducción Técnica

Este manual está dirigido a la persona encargada de instalar, configurar o mantener el sistema Gestión y TICS. No es necesario ser un programador experto para seguir este documento, pero sí se requiere conocimiento básico de computadores y redes.

El sistema está dividido en dos partes que trabajan juntas:

- El Backend (parte trasera): es el servidor que procesa toda la información, se comunica con la base de datos y responde a las solicitudes de los usuarios. Está construido con Node.js y Express.
- El Frontend (parte visible): es la pantalla que ve el usuario en el navegador. Está construido con React.js y se comunica con el backend para mostrar y guardar información.
- La Base de Datos: es donde se guarda toda la información del sistema (ofertas, usuarios, solicitudes, etc.). Se usa MongoDB, una base de datos que guarda la información de forma flexible.

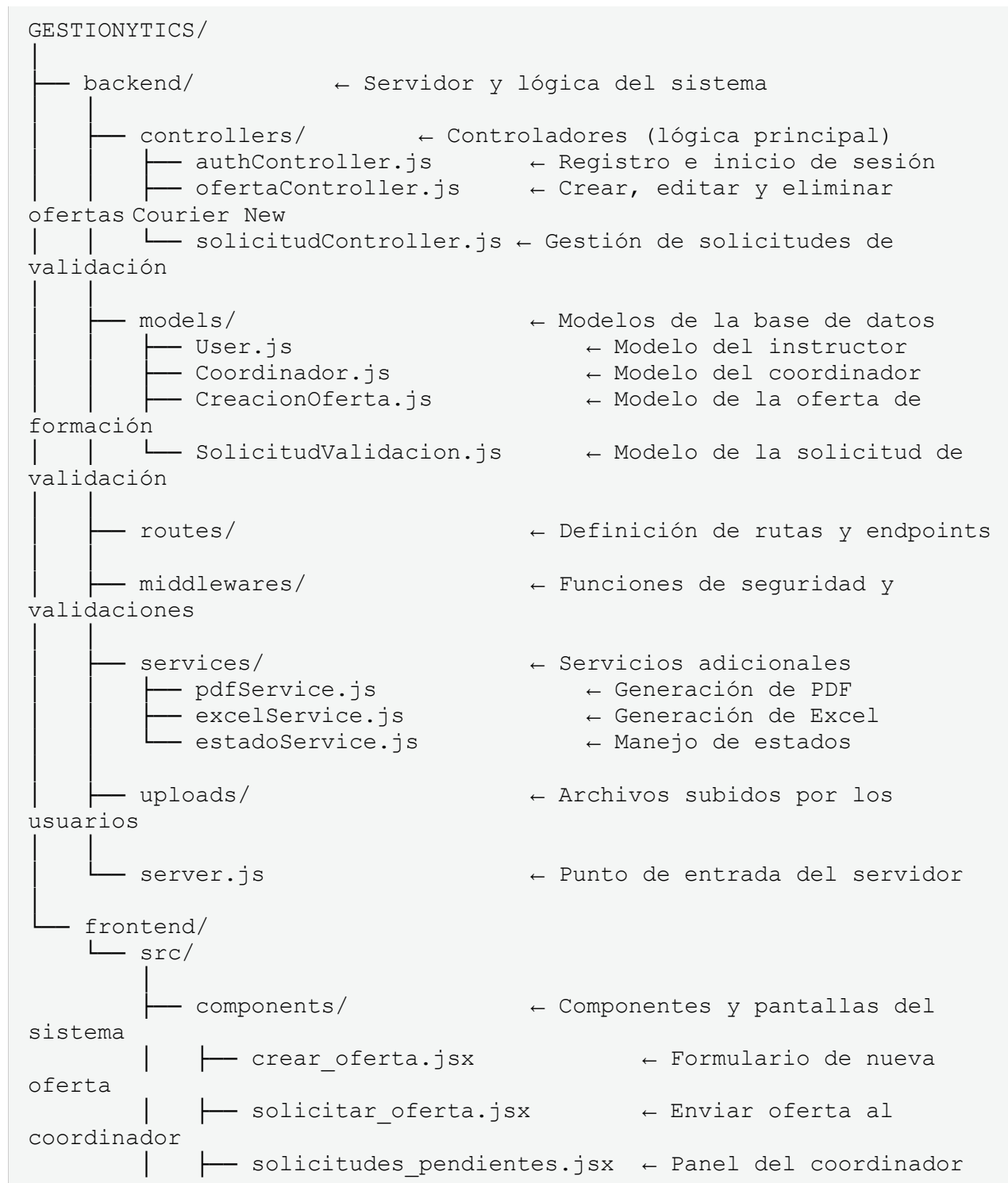
2. Tecnologías Utilizadas

A continuación, se listan las tecnologías principales del sistema y para qué sirve cada una:

Tecnología	Para qué sirve	Versión mínima
Node.js	Es el motor que hace funcionar el servidor (backend). Permite ejecutar código JavaScript fuera del navegador.	18 o superior
Express.js	Es la librería que organiza las rutas y peticiones del servidor. Define qué hace el sistema cuando recibe cada solicitud.	4.x
MongoDB	Es la base de datos donde se guarda toda la información del sistema.	6 o superior
Mongoose	Es la herramienta que conecta el servidor con MongoDB y define la estructura de los datos.	8.x
React.js	Es la tecnología con la que está construida la interfaz visual que ven los usuarios.	18 o superior
JWT	Sistema de seguridad que verifica la identidad del usuario en cada solicitud, sin necesidad de consultar la base de datos en cada momento.	9.x
bcryptjs	Herramienta que cifra las contraseñas de los usuarios para que no se guarden en texto plano.	2.x
ExcelJS	Librería para generar archivos Excel con el listado de inscritos.	4.x

3. Estructura de Archivos del Proyecto

El proyecto está organizado en dos carpetas principales: backend y frontend. A continuación se explica qué contiene cada carpeta importante:





4. Sistema de Autenticación y Seguridad

El sistema usa un mecanismo llamado JWT (JSON Web Token) para verificar que cada usuario sea quien dice ser. Funciona así:

1. El usuario ingresa su correo y contraseña en el formulario de inicio de sesión.
2. El servidor verifica las credenciales contra la base de datos.
3. Si son correctas, el servidor genera un "token" (un código único y cifrado) que identifica al usuario y contiene su tipo de rol.
4. El navegador guarda ese token y lo envía en cada solicitud posterior.
5. El servidor verifica el token en cada solicitud para saber quién es el usuario y qué permisos tiene.

El token incluye el campo "tipo" que indica si el usuario es instructor, coordinador o funcionario. Esto permite que el sistema busque al usuario en la colección correcta de la base de datos:

Tipo en el token	Colección de MongoDB donde se busca
instructor	users
coordinador	coordinadors
funcionario	funcionarios

Atención

Los tokens de sesión expiran después de 30 días. Si un usuario reporta que el sistema "no lo deja entrar" aunque su contraseña sea correcta, puede ser que su token haya expirado. La solución es cerrar sesión y volver a iniciarla.

5. Base de Datos — Colecciones Principales

MongoDB guarda la información en "colecciones" (equivalentes a las tablas en bases de datos tradicionales). Las más importantes del sistema son:

5.1 users — Instructores

Guarda la información de todos los instructores registrados en el sistema. El campo más importante para el funcionamiento del sistema es `coordinadorAsignado`, que indica a qué coordinador pertenece cada instructor.

Atención

Si el campo `coordinadorAsignado` de un instructor no contiene el ID correcto del coordinador, las solicitudes que ese instructor envíe no aparecerán en el panel del coordinador. Este es el error más frecuente del sistema.

5.2 coordinadors — Coordinadores

Guarda la información de los coordinadores. Cada coordinador tiene un identificador único (`_id`) que debe coincidir con el campo `coordinadorAsignado` de sus instructores.

5.3 creacionofertas — Ofertas de Formación

Guarda cada oferta creada por los instructores. Los campos más importantes son:

- `estado`: indica en qué punto del proceso está la oferta (borrador, pendiente, aprobada, rechazada, `ficha_creada`).
- `coordinador_asignado`: el coordinador responsable de revisar esta oferta.
- `creado_por`: el instructor que creó la oferta.
- `carta_pdf` y `firma_digital_pdf`: rutas en el servidor donde están guardados los archivos adjuntos.

5.4 solicitudvalidacions — Solicitudes de Validación

Cada vez que un instructor envía una oferta para revisión, se crea un registro en esta colección. Los campos clave son:

- oferta_id: referencia a la oferta que se está enviando a revisar.
- instructor_id: referencia al instructor que envió la solicitud.
- coordinador_id: referencia al coordinador que debe revisar. Debe coincidir con el _id del coordinador logueado.
- estado: puede ser pendiente, aprobada o rechazada.

5.5 estadoofertas — Estados del Sistema

Esta colección guarda los posibles estados de una oferta: borrador, pendiente, aprobada, rechazada y ficha_creada. Esta colección debe ser poblada antes de que el sistema pueda funcionar.

Atención

Si esta colección está vacía, el sistema no podrá crear ofertas y mostrará el error: "No se encontró el estado borrador". Ejecute el script insertarEstados.js para solucionarlo.

6. Rutas del Sistema (API)

El backend expone una serie de rutas (URLs) a las que el frontend envía solicitudes. Todas las rutas requieren que el usuario haya iniciado sesión, excepto las de registro y login.

6.1 Autenticación

Acción	Ruta	Quién la usa
Registrar usuario	POST /api/auth/register	Administrador del sistema
Iniciar sesión	POST /api/auth/login	Todos los usuarios

6.2 Ofertas

Acción	Ruta	Quién la usa
Crear oferta	POST /api/ofertas/	Instructor
Ver mis ofertas	GET /api/ofertas/mis-ofertas	Instructor
Actualizar oferta	PUT /api/ofertas/:id	Instructor
Eliminar oferta	DELETE /api/ofertas/:id	Instructor
Descargar PDF	GET /api/ofertas/:id/pdf	Instructor / Coordinador

6.3 Solicitudes de Validación

Acción	Ruta	Quién la usa
Enviar solicitud	POST /api/solicitudes/validacion	Instructor
Ver pendientes	GET /api/solicitudes/pendientes	Coordinador
Aprobar solicitud	PUT /api/solicitudes/:id/aprobar	Coordinador
Rechazar solicitud	PUT /api/solicitudes/:id/rechazar	Coordinador
Descargar ficha	GET /api/solicitudes/:id/descargar/ficha	Coordinador
Descargar Excel	GET /api/solicitudes/:id/descargar/excel	Coordinador

Nota técnica

En las rutas, ":id" es un espacio reservado que se reemplaza con el identificador real del registro. Por ejemplo: /api/solicitudes/abc123/aprobar aprueba la solicitud con ID abc123.

7. Instalación del Sistema

Siga estos pasos para instalar y poner en funcionamiento el sistema en un servidor nuevo.

7.1 Requisitos previos

Antes de instalar, verifique que el servidor cuente con:

- Node.js versión 18 o superior instalado.
- MongoDB versión 6 o superior instalado y en ejecución.
- npm (gestor de paquetes de Node) versión 9 o superior.
- Acceso a internet para descargar las dependencias.

7.2 Configurar las variables de entorno

El sistema necesita un archivo llamado .env dentro de la carpeta backend. Este archivo contiene información sensible como la contraseña de la base de datos. Cree el archivo con el siguiente contenido y ajuste los valores según su configuración:

```
MONGO_URI=mongodb://localhost:27017/gestionytics  
JWT_SECRET=escriba_aqui_una_clave_secreta_larga_y_segura PORT=5000
```

Atención

Nunca comparta el archivo .env ni lo suba a repositorios públicos como GitHub. Contiene información sensible del sistema.

7.3 Instalar y arrancar el backend

Abra una terminal, navegue hasta la carpeta del proyecto y ejecute los siguientes comandos en orden:

```
cd backend npm install node scripts/insertarEstados.js npm start
```

El comando "npm install" descarga todas las librerías necesarias. El comando "insertarEstados.js" crea los estados iniciales en la base de datos (esto solo se hace una vez). El comando "npm start" arranca el servidor.

7.4 Instalar y arrancar el frontend

En otra terminal, ejecute:

```
cd frontend npm install npm run dev
```

Si todo está bien, el sistema estará disponible en su navegador en la dirección <http://localhost:3000> (o el puerto que indique el frontend).

7.5 Datos que deben existir antes de usar el sistema

Antes de que los usuarios puedan empezar a trabajar, algunas colecciones deben tener datos. Si están vacías, el sistema no funcionará correctamente:

Colección	Por qué es necesaria	Cómo poblarla
estadoofertas	Sin los estados, no se pueden crear ofertas.	Ejecutar: node scripts/insertarEstados.js
modalidades	El formulario de creación de ofertas necesita las modalidades disponibles.	Insertar manualmente desde MongoDB Compass o con un script.
tipooferas	Define los tipos de oferta disponibles en el formulario.	Insertar manualmente o con script.
tipoprogramas	Tipos de programa de formación disponibles.	Insertar manualmente o con script.
municipios	Lista de municipios de Colombia para la ubicación de la oferta.	Importar desde CSV o script dedicado.

8. Resolución de Problemas Comunes

A continuación se describen los errores más frecuentes del sistema, su causa y cómo resolverlos:

Problema 1: Las solicitudes no aparecen en el panel del coordinador

Causa: El campo `coordinador_id` en la solicitud de validación no coincide con el `_id` del coordinador que está logueado. Esto ocurre cuando el instructor fue registrado con un coordinador incorrecto o que ya no existe.

Solución: Abra MongoDB Compass, busque el instructor en la colección `users` y verifique que su campo `coordinadorAsignado` tenga el ID correcto del coordinador. Si no coincide, actualícelo con el comando:

```
db.users.updateOne(  { _id: ObjectId("ID_DEL_INSTRUCTOR") },  { $set: {
  coordinadorAsignado: ObjectId("ID_CORRECTO_DEL_COORDINADOR") } } )
```

Problema 2: Error "No se encontró el estado borrador"

Causa: La colección `estadoofertas` está vacía. Esto ocurre cuando el sistema se instala por primera vez sin ejecutar el script de inicialización.

Solución: Ejecute en la terminal:

```
node scripts/insertarEstados.js
```

Problema 3: El usuario no puede iniciar sesión aunque su contraseña sea correcta

Posibles causas y soluciones:

- El token expiró (después de 30 días): el usuario debe cerrar sesión completamente, limpiar el caché del navegador e iniciar sesión nuevamente.
- La variable JWT_SECRET cambió en el servidor: todos los tokens existentes quedarán inválidos. Los usuarios deberán iniciar sesión nuevamente.
- El usuario fue registrado en una colección diferente a la que busca el sistema: verifique que el correo exista en la colección correcta (users para instructores, coordinadors para coordinadores).

Problema 4: Los archivos PDF o de firma digital no se pueden descargar

Causa: Los archivos se guardan en la carpeta uploads/ del servidor con la ruta absoluta del computador donde fueron subidos. Si el sistema se movió a otro servidor, las rutas ya no son válidas.

Solución a corto plazo: Verifique que la carpeta uploads/ exista en el servidor y tenga permisos de lectura y escritura. Si el sistema fue migrado, los archivos deben ser copiados al nuevo servidor manteniendo la misma estructura de carpetas.

Buena práctica

Para un sistema en producción, se recomienda migrar el almacenamiento de archivos a un servicio en la nube como Amazon S3 o Cloudinary. Esto evita que los archivos se pierdan si el servidor falla o se reinstala.

Problema 5: El sistema muestra errores en la consola del servidor

El sistema tiene registros (logs) detallados en la consola del servidor para facilitar el diagnóstico. Los mensajes están marcados con emojis:

-  — La operación fue exitosa.
-  — Ocurrió un error. El mensaje siguiente describe qué falló.
-  — El sistema está buscando o verificando algo.
-  — Resultado de una consulta a la base de datos.

Cuando ocurra un error, busque en la consola el mensaje más reciente que empiece con  y lea la descripción del error para identificar el problema.